

BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: William Natcharian

ID: WRN25

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/Desktop/4750 Cloned Repositories/hw4-wrn25\_hw4`

```

using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables

```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned}
 & \max_{x,y} && 3x + 2y \\
 & \text{subject to:} && x + 4y \leq 16 \\
 & && x - 2y \leq -1 \\
 & && 0 \leq x \leq 4 \\
 & && 1 \leq y \leq 4.
 \end{aligned}$$

→

```

x = 0:0.01:4

# Example: y <= 1 and y <= 3 - 2x/3
#uppery = max.(1.0, 3 .- 2 .* x ./ 3)
#uppery = max.(10 .+x*0, 10 .- x)
# Example: y <= 5/3 + x
#lowery = 5 .+ x

lowery = min.(4, 4 .- x*0.25)
uppery = max.(1, 0.5 .+ x*0.5)

line = 0.5 .+ x*0.5
extra = 1:0.01:4

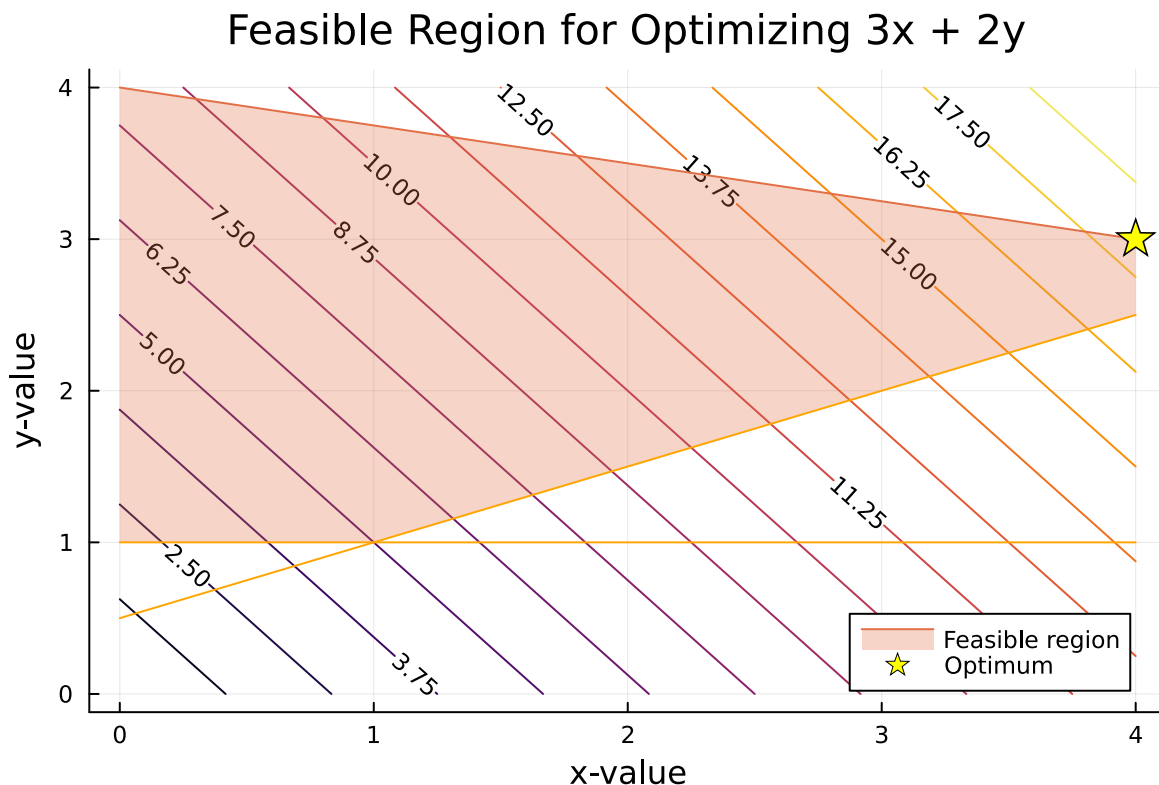
f1(x, y) = 3x + 2y
optimum = f1(4,3)
x = 0:0.01:4
y = 0:0.01:4

```

```

p1 = contour(x, y, f1, title="Feasible Region for Optimizing 3x + 2y", xlabel="x-value", ylabel="y-value")
plot(p1, x, lowery, fillrange = upperry, fillalpha = 0.3, label = "Feasible region")
plot(p1, x, line, color =:orange, label = "")
plot(p1, x, 1 + x*0, color =:orange, label = "")
scatter!(p1, [4], [3], label = "Optimum", markershape =:star5, markersize = 10, markercolor =:yellow)
display(p1)
print("Optimum solution at (4,3) with a maximum value of $ optimum")

```



Optimum solution at (4,3) with a maximum value of 18

-> The plot above shows the contour lines for the equation we are trying to maximize, $3x + 2y$, with the feasible region plotted on top of it based on the four restricting equations.

-> The solution will always be found on one of the four corners of the feasible region. In this case it is obvious from the values of the contour lines that the optimum solution is in the top right corner. This optimum solution is found at (4,3) and has a value of 18.

-> Code Explanation: I used the `contour()` plotting function to add the objective value lines behind the feasible region. To create the feasible region I used the `fillrange()` function within

the plotting function and defined the tops and bottoms by the corresponding minimums and maximum inequalities describing the region.

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear.**

Decision Variables:

- Space allocated to Soybeans in ha with pesticide application rates of 0, 1, and 2 respectively:
- S_{S0}, S_{S1}, S_{S2}
- Space allocated to Wheat in ha with pesticide application rates of 0, 1, and 2 respectively:
- S_{W0}, S_{W1}, S_{W2}
- Space allocated to Corn in ha with pesticide application rates of 0, 1, and 2 respectively:
- S_{C0}, S_{C1}, S_{C2}

Objective Function(s): To Maximize Profit per year

- $\text{Max } (S_{S0} * (2900 * 0.36 - 350 - 70 * 0) + S_{S1} * (3800 * 0.36 - 350 - 70 * 1) + S_{S2} * (4400 * 0.36 - 350 - 70 * 2) + S_{W0} * (3500 * 0.27 - 280 - 70 * 0) + S_{W1} * (4100 * 0.27 - 280 - 70 * 1) + S_{W2} * (4200 * 0.27 - 280 - 70 * 2) + S_{C0} * (5900 * 0.22 - 390 - 70 * 0) + S_{C1} * (6700 * 0.22 - 390 - 70 * 1) + S_{C2} * (7900 * 0.22 - 390 - 70 * 2))$
- $\rightarrow$
- $\text{Max } (694S_{S0} + 948S_{S1} + 1094S_{S2} + 555S_{W0} + 757S_{W1} + 714S_{W2} + 908S_{C0} + 1014S_{C1} + 1208S_{C2})$

Constraints:

- Average pesticide applications:
- $-0.8S_{S0} + 0.2S_{S1} + 1.2S_{S2} \leq 0$
- $-0.7S_{W0} + 0.3S_{W1} + 1.3S_{W2} \leq 0$
- $-0.6S_{C0} + 0.4S_{C1} + 1.4S_{C2} \leq 0$
- Total hectares used:
- $S_{S0} + S_{S1} + S_{S2} + S_{W0} + S_{W1} + S_{W2} + S_{C0} + S_{C1} + S_{C2} \leq 130$
- Total amount in kg produced:
- $2900S_{S0} + 3800S_{S1} + 4400S_{S2} \leq 250000$
- $3500S_{W0} + 4100S_{W1} + 4200S_{W2} \leq 250000$
- $5900S_{C0} + 6700S_{C1} + 7900S_{C2} \leq 250000$

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

```
plant_model = Model(HiGHS.Optimizer) # initialize model object
@variable(plant_model, S[1:9] >= 0) # non-negativity constraints
# uncomment the following lines and add the objective and constraints as needed for the model
@objective(plant_model, Max, 694S[1] + 948S[2] + 1094S[3] + 555S[4] + 757S[5] + 714S[6] + 908S[7] + 1014S[8] + 908S[9])

@constraint(plant_model, soy_average, -0.8S[1] + 0.2S[2] + 1.2S[3] <= 0)
@constraint(plant_model, wheat_average, -0.7S[4] + 0.3S[5] + 1.3S[6] <= 0)
@constraint(plant_model, corn_average, -0.6S[7] + 0.4S[8] + 1.4S[9] <= 0)

@constraint(plant_model, total_hectares, S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S[8] + S[9] <= 130)

@constraint(plant_model, total_kilos_soy, 2900S[1] + 3800S[2] + 4400S[3] <= 250000)
@constraint(plant_model, total_kilos_wheat, 3500S[4] + 4100S[5] + 4200S[6] <= 250000)
@constraint(plant_model, total_kilos_corn, 5900S[7] + 6700S[8] + 7900S[9] <= 250000)

print(plant_model) # this outputs a nicely formatted summary of the model so you can check your model
optimize!(plant_model)
```

```
Max 694 S[1] + 948 S[2] + 1094 S[3] + 555 S[4] + 757 S[5] + 714 S[6] + 908 S[7] + 1014 S[8] + 908 S[9]
Subject to
soy_average : -0.8 S[1] + 0.2 S[2] + 1.2 S[3]    0
wheat_average : -0.7 S[4] + 0.3 S[5] + 1.3 S[6]    0
corn_average : -0.6 S[7] + 0.4 S[8] + 1.4 S[9]    0
total_hectares : S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S[8] + S[9]    130
total_kilos_soy : 2900 S[1] + 3800 S[2] + 4400 S[3]    250000
total_kilos_wheat : 3500 S[4] + 4100 S[5] + 4200 S[6]    250000
total_kilos_corn : 5900 S[7] + 6700 S[8] + 7900 S[9]    250000
S[1]    0
S[2]    0
S[3]    0
S[4]    0
S[5]    0
S[6]    0
S[7]    0
S[8]    0
S[9]    0
```

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [6e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros 0s
7 rows, 9 cols, 27 nonzeros 0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration      Objective      Infeasibilities num(sum)
          0      -1.7998422848e+02 Ph1: 7(28.3601); Du: 9(179.984) 0s
          7       1.1599940331e+05 Pr: 0(0) 0s
Model status      : Optimal
Simplex iterations: 7
Objective value    : 1.1599940331e+05
P-D objective error : 6.2723824816e-17
HiGHS run time     : 0.00

```

```

@show value.(S);
@show objective_value(plant_model);
total = sum(value.(S))
print("Total hectares of space used: $total")

```

```

value.(S) = [13.812154696132604, 55.24861878453038, 0.0, 6.7433064173395625, 15.734381640458]
objective_value(plant_model) = 115999.40331491713
Total hectares of space used: 130.0

```

→ Code Explanation: I used the same JuMP code setup that we used in lab 3 to solve this problem, while filling in the new objective and constraints. For simplicity, I set the variable to all be components of S , so that I can easily refer to them throughout the rest of the problem.

→ From this JuMP analysis, we found that the optimal allocation of ha for the three crops with varying pesticide applications is as follows:

Soy:

- $S_{S0} = 13.81$ ha
- $S_{S1} = 55.25$ ha

- $S_{S2} = 0.0$ ha

Wheat:

- $S_{W0} = 6.74$ ha
- $S_{W1} = 15.73$ ha
- $S_{W2} = 0.0$ ha

Corn:

- $S_{C0} = 26.92$ ha
- $S_{C1} = 0.0$ ha
- $S_{C2} = 11.538$ ha

→

- Given these values above, the total expected profit for the farmer is \$115999.40

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

→ To answer this problem, I will use the same code as in part 2.2, but I'll change the constraint which covers the total hectares available to match the new value:

```
plant_models = Model(HiGHS.Optimizer) # initialize model object
@variable(plant_models, S[1:9] >= 0) # non-negativity constraints
# uncomment the following lines and add the objective and constraints as needed for the model
@objective(plant_models, Max, 694S[1] + 948S[2] + 1094S[3] + 555S[4] + 757S[5] + 714S[6] + 900S[7] + 550S[8] + 420S[9])

@constraint(plant_models, soy_average, -0.8S[1] + 0.2S[2] + 1.2S[3] <= 0)
@constraint(plant_models, wheat_average, -0.7S[4] + 0.3S[5] + 1.3S[6] <= 0)
@constraint(plant_models, corn_average, -0.6S[7] + 0.4S[8] + 1.4S[9] <= 0)

@constraint(plant_models, total_hectares, S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S[8] + S[9] <= 100)

@constraint(plant_models, total_kilos_soy, 2900S[1] + 3800S[2] + 4400S[3] <= 250000)
@constraint(plant_models, total_kilos_wheat, 3500S[4] + 4100S[5] + 4200S[6] <= 250000)
@constraint(plant_models, total_kilos_corn, 5900S[7] + 6700S[8] + 7900S[9] <= 250000)
```



```
print(plant_models) # this outputs a nicely formatted summary of the model so you can check y
optimize!(plant_models)
```

```
Max 694 S[1] + 948 S[2] + 1094 S[3] + 555 S[4] + 757 S[5] + 714 S[6] + 908 S[7] + 1014 S[8] -
Subject to
  soy_average : -0.8 S[1] + 0.2 S[2] + 1.2 S[3]    0
  wheat_average : -0.7 S[4] + 0.3 S[5] + 1.3 S[6]    0
  corn_average : -0.6 S[7] + 0.4 S[8] + 1.4 S[9]    0
  total_hectares : S[1] + S[2] + S[3] + S[4] + S[5] + S[6] + S[7] + S[8] + S[9]    140
  total_kilos_soy : 2900 S[1] + 3800 S[2] + 4400 S[3]    250000
  total_kilos_wheat : 3500 S[4] + 4100 S[5] + 4200 S[6]    250000
  total_kilos_corn : 5900 S[7] + 6700 S[8] + 7900 S[9]    250000
  S[1]    0
  S[2]    0
  S[3]    0
  S[4]    0
  S[5]    0
  S[6]    0
  S[7]    0
  S[8]    0
  S[9]    0
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP    has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost    [6e+02, 1e+03]
  Bound   [0e+00, 0e+00]
  RHS     [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced
Problem not reduced by presolve: solving the LP
Using EKK dual simplex solver - serial
  Iteration      Objective      Infeasibilities num(sum)
        0      -1.7998422848e+02 Ph1: 7(28.3601); Du: 9(179.984) 0s
        7       1.2296340331e+05 Pr: 0(0) 0s
Model status      : Optimal
Simplex iterations: 7
Objective value    : 1.2296340331e+05
P-D objective error : 5.9171491942e-17
```

HiGS run time : 0.00

```
@show value.(S);  
@show objective_value(plant_models);  
total = sum(value.(S))  
print("Total hectares of space used: $total")
```

```
value.(S) = [13.8121546961326, 55.24861878453038, 0.0, 9.743306417339564, 22.734381640458977  
objective_value(plant_models) = 122963.40331491713  
Total hectares of space used: 140.0
```

-> As we can see from this new analysis, buying 10 extra hectares of land would provide an additional \$6964 of profit for the farmer, assuming the optimal solutions are applied to pesticide and crop allocation.

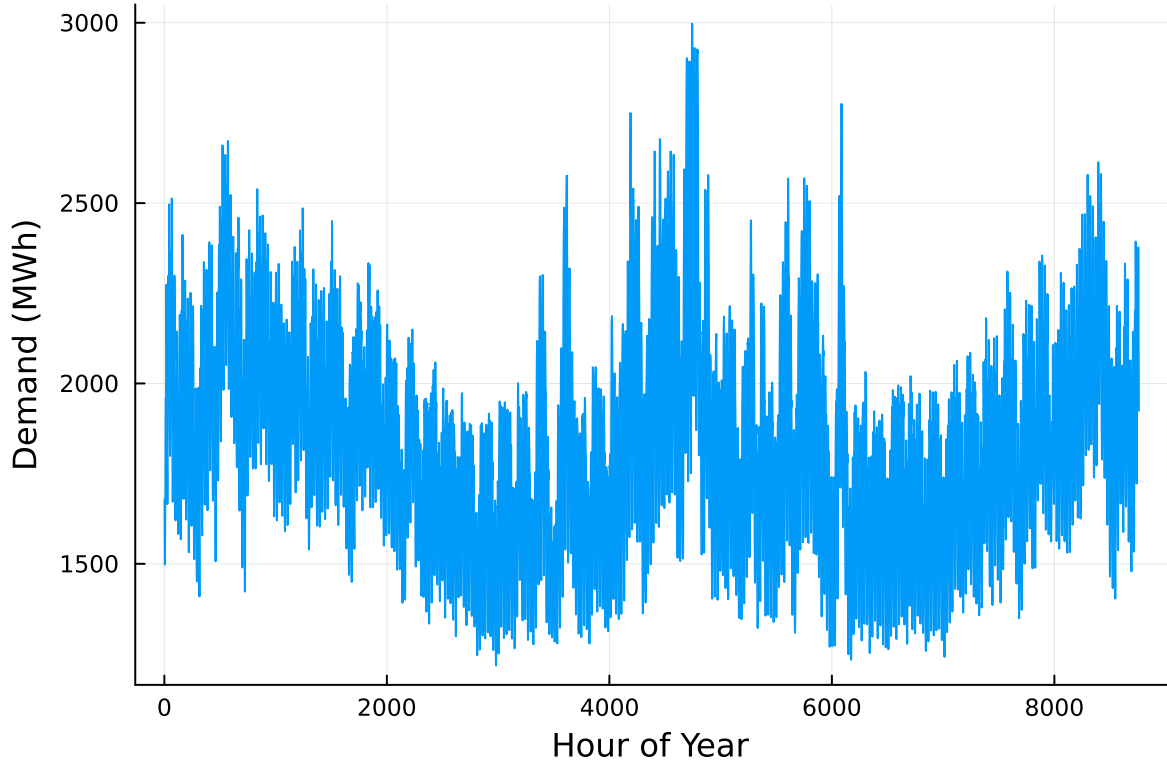
-> This means that I would only advise the farmer to adopt this plan if the cost of the 10 hectares was less than the net profit it would create for them.

-> Adding more land after having 130 ha already is not an efficient use of time, because the maximum amounts of soybeans and corn are already being produced. We know this because when increasing the max amount of hectares available, only the ha for wheat goes up. Most likely the limiting factor is the total kg limit, which the soybeans and corn have already hit at 130 ha total. Since wheat is not a very profitable crop, buying extra land to grow solely more wheat usually won't be a good economic investment.

Problem 3 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
# load the data, pull Zone C, and reformat the DataFrame  
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))  
rename!(NY_demand, : "Time Stamp" => :Date)  
demand = NY_demand[:, [:Date, :C]]  
rename!(demand, :C => :Demand)  
demand[:, :Hour] = 1:nrow(demand)  
  
# plot demand  
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs (\$/MW installed), variable costs (\$/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```

# load capacity factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)

```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO₂ limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO₂/yr.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of \$10,000/MWh.

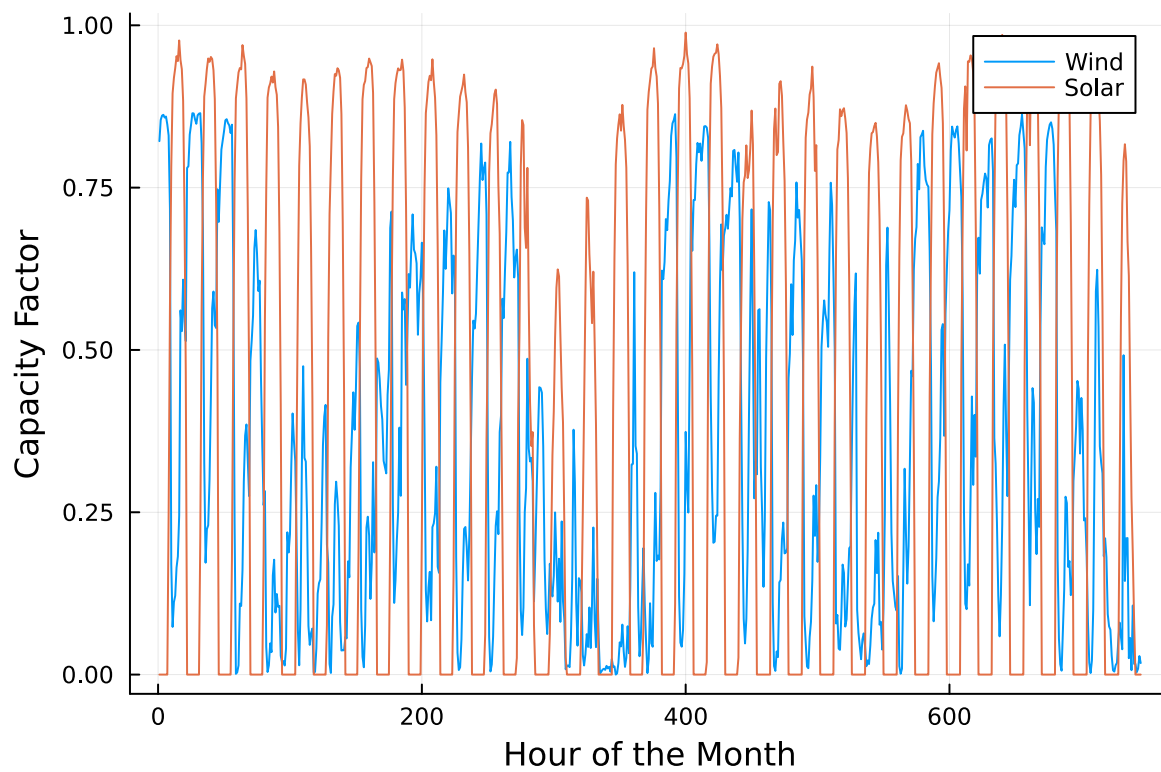
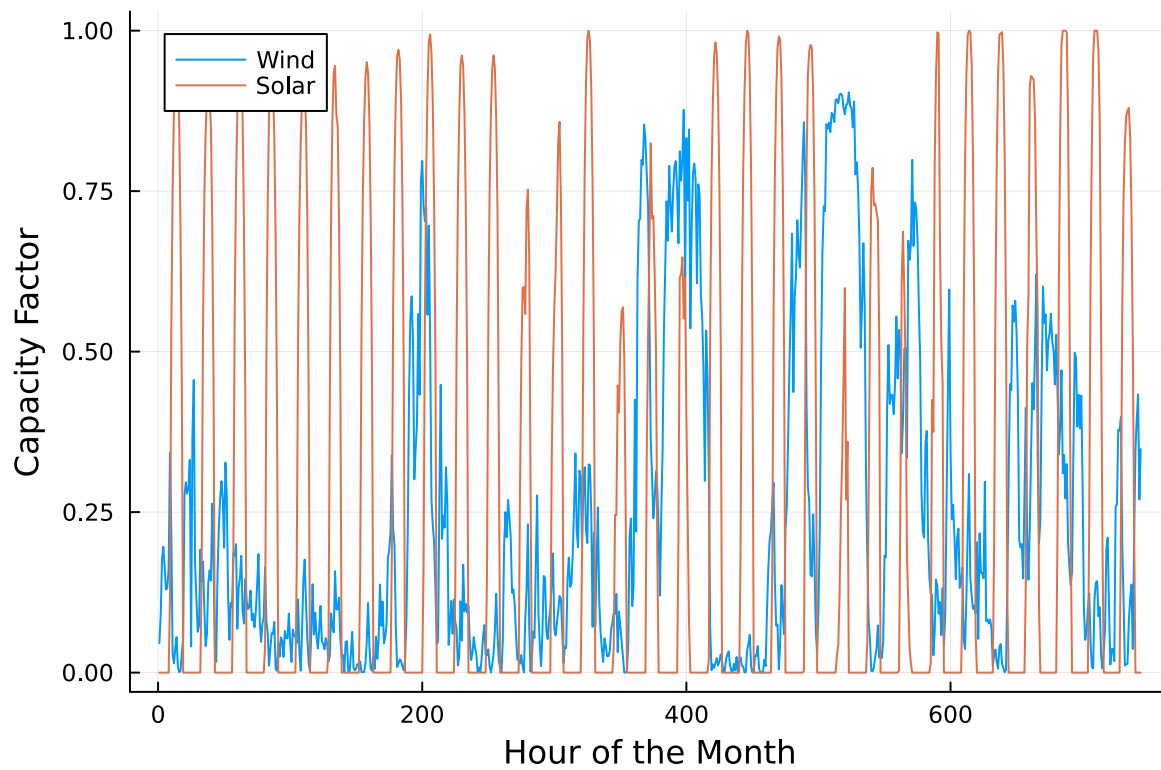
Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Objective:

- $\min \sum_i FixedCost_i X_i + \sum_i \sum_t VarCost_i g_{i,t} + \sum_i P_{NS} u_t$
- Where X_i is the installed capacity of plant type i in MW, $g_{i,t}$ is the electricity generation from plant type i during hour t in MWh, and u_t is the unserved demand during hour t in MWh.

Constraints:



- In every hour, total generation plus unmet demand equals the total demand:
- $\sum_i g_{i,t} + u_t = D_t$ (For every hour t)
- A plant can't generate more than its available capacity:
- $g_{i,t} \leq X_i * CF_{i,t}$
- Total annual emissions can't exceed the 1.5MtCO₂ cap:
- $\sum_i \sum_t Em_i * g_{i,t} \leq E^{max}$
- No negative capacities, generation, or demand:
- $X_i \geq 0$
- $g_{i,t} \geq 0$
- $u_t \geq 0$
- Where D_t is the electricity demand at hour t in MWh, $CF_{i,t}$ is the capacity factor of installed capacity available at hour t, $FixedCost_i$ is the annualized capital cost of building 1MW of capacity of type i in \$/MWyear, $VarCost_i$ is the variable operating cost of generating 1 MWh from plant type i in \$/MWh, Em_i is the CO₂ emission intensity per MWh generated by plant i in tCO₂/MWh, E^{max} is the CO₂ emission cap for the utility $\$ = 1.5 \cdot 10^6$ tCO₂/year, and P_{NS} is the penalty cost for unserved energy = \$10000/MWh

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

```
demand_df = CSV.read("data/2013_hourly_load_NY.csv", DataFrame)
if :C in propertynames(demand_df)
    rename!(demand_df, :C => :Demand)

elseif :Demand in propertynames(demand_df) && :Load in propertynames(demand_df)
    rename!(demand_df, :Load => :Demand)
end

T = nrow(demand_df) # number of hours
D = demand_df.Demand |> collect # hourly demand vector in MWh

plants = collect(gens.Plant)
n_plants = length(plants)
```

```

CF = zeros(n_plants, T)
for (i,p) in enumerate(plants)
    if occursin(r"Wind"i, string(p))
        CF[i, :] = cf_df.Wind[1:T]
    elseif occursin(r"Solar"i, string(p))
        CF[i, :] = cf_df.Solar[1:T]
    elseif occursin(r"Geothermal"i, string(p))
        CF[i, :] .= 0.85
    else
        CF[i, :] .= 1.0
    end
end

FixedCost = Dict{plants[i] => float(gens.FixedCost[i]) for i in 1:n_plants} #$/MW-year
VarCost    = Dict{plants[i] => float(gens.VarCost[i])   for i in 1:n_plants} #$/MWh
Emissions  = Dict{plants[i] => float(gens.Emissions[i]) for i in 1:n_plants} #tCO2/MWh

P_ns = 10_000.0 #$/MWh penalty for extra energy
E_cap = 1.5e6 #tCO2 per year allowed = 1.5 MtCO2

#JuMP
model = Model(HiGHS.Optimizer)
set_silent(model)

@variable(model, X[1:n_plants] >= 0) #installed capacity in MW
@variable(model, g[1:n_plants, 1:T] >= 0) #generation in MWh in each hour
@variable(model, u[1:T] >= 0) #unserved energy in MWh per hour

@objective(model, Min,
    sum(FixedCost[plants[i]] * X[i] for i in 1:n_plants) + sum(VarCost[plants[i]] * g[i,t] f
    + P_ns * sum(u[t] for t in 1:T))

for t in 1:T
    @constraint(model, sum(g[i,t] for i in 1:n_plants) + u[t] == D[t])
end

for i in 1:n_plants, t in 1:T
    @constraint(model, g[i,t] <= X[i] * CF[i,t])
end

@constraint(model, sum( Emissions[plants[i]] * g[i,t] for i in 1:n_plants, t in 1:T ) <= E_c

```

$$g_{2,1}+0.43g_{3,1}+0.55g_{4,1}+g_{2,2}+0.43g_{3,2}+0.55g_{4,2}+g_{2,3}+0.43g_{3,3}+0.55g_{4,3}+g_{2,4}+0.43g_{3,4}+0.55g_{4,4}+g_{2,5}+0.43g_{3,5}+$$

-> Code explanation: In the code above, I add the objective functions and constraints to the JuMP model using a series of if-statements and for loops to create the large number of total constraints for all the available plant types at different times. I also added several checks that the dimensions of the data from the .csv file were correct during troubleshooting to make sure everything was working.

```
optimize!(model)

X_val = value.(X)
g_val = value.(g)
u_val = value.(u)

installed = DataFrame(Plant = plants, Installed_MW = X_val, FixedCost_per_MW = [FixedCost[p]

annual_gen = [sum( g_val[i, t] for t in 1:T ) for i in 1:n_plants]

annual_gen_df = DataFrame(Plant = plants, Annual_Generation_MWh = annual_gen, VarCost_per_MWh
Emissions_tCO2_per_MWh = [Emissions[p] for p in plants])

total_fixed_cost = sum(FixedCost[plants[i]] * X_val[i] for i in 1:n_plants)
total_var_cost    = sum(VarCost[plants[i]] * annual_gen[i] for i in 1:n_plants)
total_unserved_energy = sum(u_val)
total_unserved_cost = P_ns * total_unserved_energy
total_cost = total_fixed_cost + total_var_cost + total_unserved_cost
total_emissions = sum(Emissions[plants[i]] * annual_gen[i] for i in 1:n_plants)

show(installed, allcols=true)
println()
println()
show(annual_gen_df, allcols=true)
println()
```

6×3 DataFrame

Row	Plant String15	Installed_MW Float64	FixedCost_per_MW Float64
1	Geothermal	285.568	450000.0

2	Coal	0.0	220000.0
3	NG CCGT	1509.17	82000.0
4	NG CT	703.062	65000.0
5	Wind	2548.89	91000.0
6	Solar	2103.75	70000.0

6×4 DataFrame

Row	Plant String15	Annual_Generation_MWh Float64	VarCost_per_MWh Float64	Emissions_tCO2_per_MWh Float64
1	Geothermal	1.097e6	0.0	0.0
2	Coal	0.0	24.0	1.0
3	NG CCGT	3.32277e6	30.0	0.43
4	NG CT	1.29473e5	40.0	0.55
5	Wind	6.45134e6	0.0	0.0
6	Solar	5.367e6	0.0	0.0

-> Based on the tables above, the utility should build no coal due to the CO₂ cap and its high cost. They should build significant CCGT and CT gas for more dispatchable power. Wind and solar should dominate installed capacity because they are zero-emissions and cost-effective in fixed terms. And A small geothermal plant should be installed due to its high capacity factor and zero emissions.

-> Total Fixed Cost = \$677,049,000 ; Total Variable Cost = \$104,862,200

-> Non-served energy = 0

-> Code explanation: I finished optimizing the JuMP model and formatted all the data points into a data fram to be easier to understand. I set allcols to true so that all available columns are shown in the data frame.

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

```
# Totals
total_capacity = sum(installed.Installed_MW)
total_generation = sum(annual_gen_df.Annual_Generation_MWh)

installed[:, :Capacity_Share] = installed.Installed_MW ./ total_capacity
annual_gen_df[:, :Generation_Share] = annual_gen_df.Annual_Generation_MWh ./ total_generation
```

```
println("Fractional contributions by plant:")
fraction_df = innerjoin(installed[:, [:Plant, :Capacity_Share]], annual_gen_df[:, [:Plant, :
show(fraction_df, allcols=true)
```

Fractional contributions by plant:

6×3 DataFrame

Row	Plant String15	Capacity_Share Float64	Generation_Share Float64
1	Geothermal	0.0399371	0.0670225
2	Coal	0.0	0.0
3	NG CCGT	0.21106	0.203009
4	NG CT	0.0983243	0.00791036
5	Wind	0.356466	0.394154
6	Solar	0.294213	0.327904

→ Fractions of annual generation: Wind and solar together produce about 72% of total electricity. CCGT provides about 20% as flexible dispatchable generation. Geothermal produces about 7%, slightly more than its share of installed capacity. NG CT contributes very little (~0.8%) because it is only used occasionally for peak demand. Coal produces nothing due to the CO₂ cap and cost (0%).

→ Comparison to Problem 3.2: Geothermal has a higher capacity factor (85%) and produces more energy relative to the installed MW. NG CT is used more sparingly for peak demand and has a lower generation relative to installed MW. Wind and Solar contribute more energy than their fraction of capacity because the model installed them where capacity factors are fairly high. And NG CCGT is flexible as the generation fraction roughly matches the capacity fraction.

→ Do these results make sense: Yes, these results are consistent with the data and constraints: The utility must meet 8,760 hours of demand in Zone C. There is a CO₂ emissions cap (1.5 Mt/yr), which prevents building coal. Wind and solar are zero-emission when installed extensively to reduce total cost under the CO₂ cap. NG CCGT is built as a dispatchable baseload to fill gaps when renewables cannot meet demand. NG CT is expensive and has a high variable cost to be only used for peak demand which explains its very low generation share. Geothermal is limited to an 85% capacity factor as it has a small capacity installed but with high utilization. This leads to a slightly higher generation fraction compared to its capacity fraction, which makes sense.

→ Code Explanation: I continued manipulating the data from the previous parts of this problem. And I used the innerjoin() function to properly format everything into columns.

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

→ Notified that we do not have not learned the information necessary to solve this part of the problem yet.

Problem 3.5

What is the shadow price of CO₂? What is the value to the utility be of allowing it to emit an additional 1000 tCO₂/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a JuMP variable `x`, but it will return a special container which holds other information about `x` that is useful for JuMP. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

```
optimize!(model)

shadow_price = JuMP.shadow_price(emissions_constr)

rhs = E_cap
lhs_val = sum(Emissions[plants[i]] * sum(value(g[i, t]) for t in 1:T) for i in 1:n_plants)
slack = rhs - lhs_val

delta = 1000.0
value_1000 = shadow_price * delta

println("CO2 constraint RHS (cap): ", rhs)
println("Total emissions at optimum (lhs): ", round(lhs_val; digits=3))
```

```
println("Slack (RHS - LHS): ", round(slack; digits=3))
println("Shadow price of CO2 (JuMP.shadow_price) = ", round(shadow_price; sigdigits=4), " \$/tCO2")
println("Value to utility of +$(Int(delta)) tCO2/yr = ", round(value_1000; digits=2), " \$/yr")
```

```
CO2 constraint RHS (cap): 1.5e6
Total emissions at optimum (lhs): 1.5e6
Slack (RHS - LHS): -0.0
Shadow price of CO2 (JuMP.shadow_price) = 0.0 \$/tCO2
Value to utility of +1000 tCO2/yr = 0.0 \$/yr
```

-> Shadow price of CO2: 0.0 \$/tCO2

-> Value to the utility be of allowing it to emit an additional 1000 tCO2/yr: 0.0 \$/yr

- Since the shadow price is zero, allowing an extra 1,000 tCO /year would not reduce the utility's costs. This means that the constraint is not limiting their operation, so there's no economic benefit to increasing emissions

-> Code Explanation: I continued using the data from previous parts of this problem and calculated the shadow price using the built in function in JuMP.

References

List any external references consulted, including classmates.

-> I worked alone; referenced online forums on how to use JuMP and the DataFrames package more efficiently.